Computational Physics Ex_3

Student ID: 2024141230044

Name: 范奕

Problem 2

Problem 2: Write a function to compute the following sequence:

$$a_0 = \frac{1}{e}(e-1)$$

$$a_{n+1} = 1-(n+1)a_n, \quad \text{for } n = 0,1,\ldots,N,$$

where N is assigned.

Compare the numerical result with **the exact limit** $a_n \to 0$ for $n \to \infty$, and **explain why**?

Hint: Rounding errors propagation happens for $n \geq 17$, and the amplified error is $\simeq 1$. When $n = 20$ the amplified error is $\simeq 30$, and so on.

目录

1 Part 1. Source Code 3

2 Part 2. Basic Principles 11

3 Part 3. Displayed Results 14

4 Part 4. Discussion 18

1 Part 1. Source Code

```
#include <iostream>
#include <iomanip>
#include <cmath>
#include <vector>
#include <string>
#include <algorithm>
#include <cstdint>

using namespace std;

// ==============================================================
// 理论值: a_n = ∫_0^1 x^n e^(x-1) dx
// 用 Simpson 积分
// ==============================================================
long double exactValueIntegral(int n) {
   const int M = 200000;
   long double h = 1.0L / M;
   long double sum = 0.0L;

   for (int i = 0; i <= M; i++) {
      long double x = i * h;
      long double fx = powl(x, n) * expl(x - 1.0L);

      if (i == 0 || i == M)
         sum += fx;
      else if (i % 2 == 0)
         sum += 2.0L * fx;
      else
         sum += 4.0L * fx;
   }

   return sum * h / 3.0L;
}

// ==============================================================
// 原程序真实递推 (double)
// ==============================================================
vector<double> computeRealRecursion(int N) {
   vector<double> a(N + 1);
```

```
   a[0] = (exp(1.0) - 1.0) / exp(1.0);

   for (int n = 0; n < N; ++n) {
      a[n + 1] = 1.0 - (n + 1) * a[n];
   }
   return a;
}

// ==============================================================
// 非负大整数, base = 1e9
// ==============================================================
struct BigUInt {
   static const uint32_t BASE = 1000000000U;
   vector<uint32_t> d; // little-endian

   void trim() {
      while (!d.empty() && d.back() == 0) d.pop_back();
   }

   bool isZero() const {
      return d.empty();
   }

   static BigUInt fromUint64(unsigned long long x) {
      BigUInt a;
      if (x == 0) return a;
      while (x) {
         a.d.push_back((uint32_t)(x % BASE));
         x /= BASE;
      }
      return a;
   }

   void mulSmall(uint32_t m) {
      if (isZero() || m == 0) {
         d.clear();
         return;
      }
      if (m == 1) return;

      unsigned long long carry = 0;
```

```
    for (size_t i = 0; i < d.size(); ++i) {
        unsigned long long cur = 1ULL * d[i] * m + carry;
        d[i] = (uint32_t)(cur % BASE);
        carry = cur / BASE;
    }
    while (carry) {
        d.push_back((uint32_t)(carry % BASE));
        carry /= BASE;
    }
}

uint32_t divSmall(uint32_t m) {
    unsigned long long rem = 0;
    for (int i = (int)d.size() - 1; i >= 0; --i) {
        unsigned long long cur = rem * BASE + d[i];
        d[i] = (uint32_t)(cur / m);
        rem = cur % m;
    }
    trim();
    return (uint32_t)rem;
}

string toDecimalString() const {
    if (isZero()) return "0";
    BigUInt tmp = *this;
    vector<uint32_t> parts;

    while (!tmp.isZero()) {
        parts.push_back(tmp.divSmall(BASE));
    }

    string s = to_string(parts.back());
    for (int i = (int)parts.size() - 2; i >= 0; --i) {
        string t = to_string(parts[i]);
        s += string(9 - t.size(), '0') + t;
    }
    return s;
}

// 仅用于显示近似值
long double toLongDoubleApprox() const {
```

```
      if (isZero()) return 0.0L;

      string s = toDecimalString();
      int take = min((int)s.size(), 18);

      long double mant = 0.0L;
      for (int i = 0; i < take; ++i) {
         mant = mant * 10.0L + (s[i] - '0');
      }

      int exp10 = (int)s.size() - take;
      return mant * powl(10.0L, exp10);
   }
};

// ==============================================================
// 有符号大整数
// ==============================================================
struct SignedBigInt {
   int sign; // -1, 0, +1
   BigUInt mag;

   SignedBigInt() : sign(0) {}

   bool isZero() const {
      return sign == 0 || mag.isZero();
   }

   static SignedBigInt fromScaledLongDouble(long double x, int SHIFT
      ) {
      SignedBigInt a;
      if (x == 0.0L) return a;

      long double scaled = ldexpl(x, SHIFT); // x * 2^SHIFT
      long double rounded = roundl(scaled);

      if (rounded == 0.0L) return a;

      a.sign = (rounded > 0.0L ? 1 : -1);
      unsigned long long v = (unsigned long long) llroundl(fabsl(
         rounded));
```

```
        a.mag = BigUInt::fromUint64(v);
        return a;
    }

    void mulSmallSigned(int k) {
        if (isZero() || k == 0) {
            sign = 0;
            mag.d.clear();
            return;
        }
        if (k < 0) {
            sign = -sign;
            k = -k;
        }
        mag.mulSmall((uint32_t)k);
        if (mag.isZero()) sign = 0;
    }

    // 仅用于显示近似值: 先转 long double 再除 2^SHIFT
    long double toApproxError(int SHIFT) const {
        if (isZero()) return 0.0L;
        long double v = mag.toLongDoubleApprox();
        if (sign < 0) v = -v;
        return ldexpl(v, -SHIFT);
    }

    string toScientificString(int SHIFT, int sigDigits = 12) const {
        long double v = toApproxError(SHIFT);
        ostringstream oss;
        oss << scientific << setprecision(sigDigits) << v;
        return oss.str();
    }
};

int main() {
    int N;
    cout << "请输入 N: ";
    cin >> N;

    // 这个值可调:
    // M_n = round(e_n * 2^SHIFT)
```

```
// SHIFT 越大，初值误差保留越细
const int SHIFT = 80;

// ------------------------------
// 理论值
// ------------------------------
vector<long double> a_theory(N + 1);
a_theory[0] = 1.0L - 1.0L / expl(1.0L);
for (int n = 1; n <= N; ++n) {
   a_theory[n] = exactValueIntegral(n);
}

// ------------------------------
// 原程序实际结果
// ------------------------------
vector<double> a_real = computeRealRecursion(N);

vector<long double> real_err(N + 1);
for (int n = 0; n <= N; ++n) {
   real_err[n] = (long double)a_real[n] - a_theory[n];
}

// ------------------------------
// 误差模型：
// e_n ≈ (-1)^n n! e0
//
// 这里 e0 取“原程序实际初值”与“理论初值”的差：
// e0 = a0_real - a0_theory
//
// 内部不直接存 e_n，而存
// M_n = round(e_n * 2^SHIFT)
// 于是：
// M_{n+1} = -(n+1) M_n
// ------------------------------
long double a0_theory = a_theory[0];
double a0_real = a_real[0];
long double e0 = (long double)a0_real - a0_theory;

vector<SignedBigInt> pred(N + 1);
pred[0] = SignedBigInt::fromScaledLongDouble(e0, SHIFT);
```

```
for (int n = 0; n < N; ++n) {
   pred[n + 1] = pred[n];
   pred[n + 1].mulSmallSigned(-(n + 1));
}

// ------------------------------
// Part 1
// ------------------------------
cout << "\n==================== Part 1 ====================\n";
cout << "原程序递推值、理论值、真实误差\n\n";

cout << scientific << setprecision(10);
cout << setw(5) << "n"
    << setw(20) << "a_n(real)"
    << setw(24) << "a_n(theory)"
    << setw(24) << "real_error"
    << "\n";

for (int n = 0; n <= N; ++n) {
   cout << setw(5) << n
       << setw(20) << a_real[n]
       << setw(24) << a_theory[n]
       << setw(24) << real_err[n]
       << "\n";
}

// ------------------------------
// Part 2
// ------------------------------
cout << "\n==================== Part 2 ====================\n";
cout << "误差模型: e_n ≈ (-1)^n n! e0\n";
cout << "其中 e0 = a0_real - a0_theory\n";
cout << "为了避免浮点爆炸, 内部采用: \n";
cout << "M_n = round(e_n * 2^SHIFT), SHIFT = " << SHIFT << "\n";
cout << "然后精确递推: M_(n+1) = -(n+1) M_n\n";
cout << "输出时再除以 2^SHIFT。\n\n";

cout << scientific << setprecision(18);
cout << "a0_theory = " << a0_theory << "\n";
cout << "a0_real = " << a0_real << "\n";
cout << "e0 = " << e0 << "\n";
```

```
    cout << "pred[0] = " << pred[0].toScientificString(SHIFT, 12) <<
       "\n\n";

    cout << scientific << setprecision(10);
    cout << setw(5) << "n"
        << setw(24) << "predicted_error"
        << "\n";

    for (int n = 0; n <= N; ++n) {
       cout << setw(5) << n
           << setw(24) << pred[n].toApproxError(SHIFT)
           << "\n";
    }

    // ----------------------------
    // Part 3
    // ----------------------------
    cout << "\n==================== Part 3 ====================\n";
    cout << "将预测误差与真实误差比较\n\n";

    cout << setw(5) << "n"
        << setw(24) << "real_error"
        << setw(24) << "predicted_error"
        << setw(24) << "difference"
        << "\n";

    for (int n = 0; n <= N; ++n) {
       long double est = pred[n].toApproxError(SHIFT);
       long double diff = real_err[n] - est;

       cout << setw(5) << n
           << setw(24) << real_err[n]
           << setw(24) << est
           << setw(24) << diff
           << "\n";
    }

    return 0;
}
```

2 Part 2. Basic Principles

This program studies the sequence

$$a_0 = 1 - \frac{1}{e}, \tag{1}$$

and

$$a_{n+1} = 1 - (n+1)a_n, \qquad n = 0, 1, 2, \ldots, N. \tag{2}$$

The main purpose is not only to compute the recurrence numerically, but also to compare the direct floating-point recursion with the theoretical behavior of the exact sequence and with a separate prediction of the propagated round-off error.

Mathematical Background

A useful exact representation of the sequence is

$$a_n = \int_0^1 x^n e^{x-1}\, dx. \tag{3}$$

By integration by parts,

$$\int_0^1 x^{n+1} e^{x-1}\, dx = \left[x^{n+1} e^{x-1}\right]_0^1 - (n+1) \int_0^1 x^n e^{x-1}\, dx, \tag{4}$$

so we obtain

$$a_{n+1} = 1 - (n+1)a_n. \tag{5}$$

Therefore the recurrence in the assignment is fully consistent with the integral formula used in the program.

The exact limit is

$$a_n \to 0 \qquad (n \to \infty), \tag{6}$$

because for $x \in [0, 1]$ we have $0 \le e^{x-1} \le 1$ and hence

$$0 \le a_n = \int_0^1 x^n e^{x-1}\, dx \le \int_0^1 x^n\, dx = \frac{1}{n+1} \to 0. \tag{7}$$

However, the direct forward recursion is numerically unstable, so the computed values in finite precision may eventually depart completely from the true limit.

Methodology

The submitted C++ program contains three logically different computations.

1. **Direct recursion in double precision.**
   The function `computeRealRecursion(int N)` stores the sequence in a `vector<double>` and computes

$$a_{n+1}^{(\text{real})} = 1 - (n+1)a_n^{(\text{real})}. \tag{8}$$

   This part reproduces the straightforward algorithm that one would naturally write first.

2. **Reference values from numerical quadrature.**
   In order to compare the floating-point result with a much more reliable reference, the function `exactValueIntegral(int n)` evaluates

$$a_n^{(\text{theory})} = \int_0^1 x^n e^{x-1}\, dx \tag{9}$$

   by composite Simpson integration with $M = 200000$ subintervals and the type `long double`. This is a distinctive feature of the code: instead of using the same recurrence for the reference solution, the program constructs the theoretical value independently from the integral formula, which avoids circular verification.

3. **Explicit error propagation model.**
   Let

$$e_n = a_n^{(\text{real})} - a_n. \tag{10}$$

   If we temporarily ignore the newly generated round-off term at each step, then the recurrence implies

$$e_{n+1} \approx -(n+1)e_n, \tag{11}$$

   and therefore

$$e_n \approx (-1)^n n! e_0. \tag{12}$$

   The program implements this model separately and prints the predicted error together with the actual error.

Special Design of the Program

The most interesting part of the implementation is the way the error model is stored. Since the quantity $(-1)^n n! e_0$ grows extremely fast, it would soon overflow if it were propagated directly as a standard floating-point number. To avoid this problem, the program defines two custom integer types:

- `BigUInt`: a nonnegative big integer with base 10^9;
- `SignedBigInt`: a signed wrapper built on top of `BigUInt`.

Instead of storing e_n itself, the code stores a scaled integer

$$M_n = \text{round}(e_n 2^{\texttt{SHIFT}}), \qquad \texttt{SHIFT} = 80. \tag{13}$$

Then the error model is propagated by the exact integer recursion

$$M_{n+1} = -(n+1)M_n. \tag{14}$$

Only when the result is printed does the program convert back by dividing by $2^{\texttt{SHIFT}}$. This design is very suitable for the present homework, because it preserves the amplification law of the error without introducing additional instability into the prediction itself.

Program Flow

The complete workflow of the program can be summarized as follows:

1. read the assigned integer N;
2. compute $a_n^{(\text{theory})}$ from the integral representation using Simpson's rule;
3. compute $a_n^{(\text{real})}$ by the original forward recurrence in `double`;
4. evaluate the true error

$$\text{real_err}_n = a_n^{(\text{real})} - a_n^{(\text{theory})}; \tag{15}$$

5. construct the predicted error sequence from e_0 using the scaled big-integer recursion;
6. print three parts of output: direct results, predicted error, and comparison between the two.

The computational cost of the direct recurrence is $O(N)$, while the cost of generating the theoretical reference is dominated by the numerical integration. Since the quadrature uses a fixed large number of subintervals, the program favors reliability of comparison over raw efficiency, which is a reasonable choice for this report-style assignment.

3 Part 3. Displayed Results

This section presents representative outputs obtained from the submitted program with input $N = 30$.

Representative Terminal Output

The first part of the program prints the recursively computed values, the reference values obtained from the integral, and the true error. A typical excerpt is listed below.

```
Input N = 30

n a_n(real) a_n(theory) real_error
16 5.5459301730e-02 5.5719345931e-02 -2.6004420167e-04
17 5.7191870597e-02 5.2771119169e-02 4.4207514283e-03
18 -2.9453670752e-02 5.0119854958e-02 -7.9573525710e-02
19 1.5596197443e+00 4.7722755796e-02 1.5118969885e+00
20 -3.0192394886e+01 4.5544884076e-02 -3.0237939770e+01
```

The table clearly shows that the direct recursion is still reasonable for small n, but around $n = 17$ the error starts to grow rapidly, and by $n = 20$ the computed value has completely lost the correct magnitude and sign.

Observed Numerical Results

Table 1 summarizes several representative values extracted from the program output. The selected rows highlight the transition from a stable-looking phase to a catastrophic error-amplification phase.

表 1: Representative numerical results for $N = 30$

n	$a_n^{(\text{real})}$	$a_n^{(\text{theory})}$	$a_n^{(\text{real})} - a_n^{(\text{theory})}$
10	$8.3877070058 \times 10^{-2}$	$8.3877070103 \times 10^{-2}$	$-4.5101461377 \times 10^{-11}$
15	$5.9033793642 \times 10^{-2}$	$5.9017540879 \times 10^{-2}$	$1.6252762604 \times 10^{-5}$
17	$5.7191870597 \times 10^{-2}$	$5.2771119169 \times 10^{-2}$	$4.4207514283 \times 10^{-3}$
18	$-2.9453670752 \times 10^{-2}$	$5.0119854958 \times 10^{-2}$	$-7.9573525710 \times 10^{-2}$
20	$-3.0192394886 \times 10^{1}$	$4.5544884076 \times 10^{-2}$	$-3.0237939770 \times 10^{1}$
25	$1.9278500883 \times 10^{8}$	$3.7086214424 \times 10^{-2}$	$1.9278500880 \times 10^{8}$
30	$-3.2967624556 \times 10^{15}$	$3.1279673932 \times 10^{-2}$	$-3.2967624556 \times 10^{15}$

The exact sequence decreases slowly toward zero, whereas the directly computed sequence eventually oscillates with enormous amplitude. This behavior is the main numerical phenomenon that the homework asks us to explain.

Error Prediction Output

The second part of the code prints the predicted error generated from

$$e_n \approx (-1)^n n! e_0. \tag{16}$$

For the present run, the initial error is approximately

$$e_0 = a_0^{(\text{real})} - a_0^{(\text{theory})} \approx -1.2468 \times 10^{-17}. \tag{17}$$

Although this number is tiny, multiplication by $n!$ makes it dominant after only a moderate number of steps. For example, the predicted error is already of order 10^{-1} near $n = 18$, of order 10^{1} near $n = 20$, and becomes astronomically large afterwards. This is consistent with the actual behavior observed in the direct recursion.

Program Screenshots

The following pictures show the output of the program, with N inputted 30.

```
platelet@platelet-YOGA-Pro-14s-IMH9:~/桌面/comp_phys/ex3$ g++ test9.cpp -o main.exe
platelet@platelet-YOGA-Pro-14s-IMH9:~/桌面/comp_phys/ex3$ ./main.exe
请输入 N: 30

==================== Part 1 ====================
原程序递推值、理论值、真实误差

    n          a_n(real)            a_n(theory)             real_error
    0    6.3212055883e-01        6.3212055883e-01       -1.2468324984e-17
    1    3.6787944117e-01        3.6787944117e-01        1.1248597540e-17
    2    2.6424111766e-01        2.6424111766e-01       -2.3771132632e-17
    3    2.0727664703e-01        2.0727664703e-01        7.3901930582e-17
    4    1.7089341189e-01        1.7089341189e-01       -2.9819625501e-16
    5    1.4553294057e-01        1.4553294057e-01        1.4914285085e-15
    6    1.2680235656e-01        1.2680235656e-01       -8.9487065761e-15
    7    1.1238350407e-01        1.1238350407e-01        6.2640702087e-14
    8    1.0093196745e-01        1.0093196745e-01       -5.0112762247e-13
    9    9.1612292994e-02        9.1612292990e-02        4.5101463593e-12
   10    8.3877070058e-02        8.3877070103e-02       -4.5101461377e-11
   11    7.7352229359e-02        7.7352228863e-02        4.9611607456e-10
   12    7.1773247695e-02        7.1773253648e-02       -5.9533928953e-09
   13    6.6947779970e-02        6.6947702576e-02        7.7394107639e-08
   14    6.2731080424e-02        6.2732163941e-02       -1.0835175069e-06
   15    5.9033793642e-02        5.9017540879e-02        1.6252762604e-05
   16    5.5459301730e-02        5.5719345931e-02       -2.6004420167e-04
   17    5.7191870597e-02        5.2771119169e-02        4.4207514283e-03
   18   -2.9453670752e-02        5.0119854958e-02       -7.9573525710e-02
   19    1.5596197443e+00        4.7722755796e-02        1.5118969885e+00
   20   -3.0192394886e+01        4.5544884076e-02       -3.0237939770e+01
   21    6.3504029260e+02        4.3557434408e-02        6.3499673516e+02
   22   -1.3969886437e+04        4.1736443028e-02       -1.3969928174e+04
   23    3.2130838805e+05        4.0061810357e-02        3.2130834799e+05
   24   -7.7114003133e+06        3.8516551423e-02       -7.7114003518e+06
   25    1.9278500883e+08        3.7086214424e-02        1.9278500880e+08
   26   -5.0124102286e+09        3.5758424983e-02       -5.0124102287e+09
   27    1.3533507617e+11        3.4522525465e-02        1.3533507617e+11
   28   -3.7893821329e+12        3.3369286982e-02       -3.7893821329e+12
   29    1.0989208185e+14        3.2290677536e-02        1.0989208185e+14
   30   -3.2967624556e+15        3.1279673932e-02       -3.2967624556e+15
```

```
==================== Part 2 ====================
误差模型: e_n ≈ (-1)^n n! e0
其中 e0 = a0_real - a0_theory
为了避免浮点爆炸, 内部采用:
M_n = round(e_n * 2^SHIFT), SHIFT = 80
然后精确递推: M_(n+1) = -(n+1) M_n
输出时再除以 2^SHIFT。

a0_theory = 6.321205588285576784e-01
a0_real   = 6.321205588285576660e-01
e0        = -1.246832498358330099e-17
pred[0]   = -1.246832498358e-17

    n         predicted_error
    0       -1.2468324984e-17
    1        1.2468324984e-17
    2       -2.4936649967e-17
    3        7.4809949901e-17
    4       -2.9923979961e-16
    5        1.4961989980e-15
    6       -8.9771939882e-15
    7        6.2840357917e-14
    8       -5.0272286334e-13
    9        4.5245057700e-12
   10       -4.5245057700e-11
   11        4.9769563470e-10
   12       -5.9723476165e-09
   13        7.7640519014e-08
   14       -1.0869672662e-06
   15        1.6304508993e-05
   16       -2.6087214389e-04
   17        4.4348264461e-03
   18       -7.9826876029e-02
   19        1.5167106446e+00
   20       -3.0334212891e+01
   21        6.3701847071e+02
   22       -1.4014406356e+04
   23        3.2233134618e+05
   24       -7.7359523084e+06
   25        1.9339880771e+08
   26       -5.0283690004e+09
   27        1.3576596301e+11
```

```
   22       -1.4014406356e+04
   23        3.2233134618e+05
   24       -7.7359523084e+06
   25        1.9339880771e+08
   26       -5.0283690004e+09
   27        1.3576596301e+11
   28       -3.8014469643e+12
   29        1.1024196197e+14
   30       -3.3072588590e+15

=================== Part 3 ===================
将预测误差与真实误差比较

    n              real_error         predicted_error              difference
    0       -1.2468324984e-17       -1.2468324984e-17        0.0000000000e+00
    1        1.1248597540e-17        1.2468324984e-17       -1.2197274440e-18
    2       -2.3771132632e-17       -2.4936649967e-17        1.1655173354e-18
    3        7.3901930582e-17        7.4809949901e-17       -9.0801931946e-19
    4       -2.9819625501e-16       -2.9923979961e-16        1.0435445910e-18
    5        1.4914285085e-15        1.4961989980e-15       -4.7704895589e-18
    6       -8.9487065761e-15       -8.9771939882e-15        2.8487412082e-17
    7        6.2640702087e-14        6.2840357917e-14       -1.9965583006e-16
    8       -5.0112762247e-13       -5.0272286334e-13        1.5952408665e-15
    9        4.5101463593e-12        4.5245057700e-12       -1.4359410742e-14
   10       -4.5101461377e-11       -4.5245057700e-11        1.4359632325e-13
   11        4.9611607456e-10        4.9769563470e-10       -1.5795601453e-12
   12       -5.9533928953e-09       -5.9723476165e-09        1.8954721148e-11
   13        7.7394107639e-08        7.7640519014e-08       -2.4641137531e-10
   14       -1.0835175069e-06       -1.0869672662e-06        3.4497592555e-09
   15        1.6252762604e-05        1.6304508993e-05       -5.1746388835e-08
   16       -2.6004420167e-04       -2.6087214389e-04        8.2794222136e-07
   17        4.4207514283e-03        4.4348264461e-03       -1.4075017763e-05
   18       -7.9573525710e-02       -7.9826876029e-02        2.5335031973e-04
   19        1.5118969885e+00        1.5167106446e+00       -4.8136560750e-03
   20       -3.0237939770e+01       -3.0334212891e+01        9.6273121499e-02
   21        6.3499673516e+02        6.3701847071e+02       -2.0217355515e+00
   22       -1.3969928174e+04       -1.4014406356e+04        4.4478182133e+01
   23        3.2130834799e+05        3.2233134618e+05       -1.0229981891e+03
   24       -7.7114003518e+06       -7.7359523084e+06        2.4551956537e+04
   25        1.9278500880e+08        1.9339880771e+08       -6.1379891343e+05
   26       -5.0124102287e+09       -5.0283690004e+09        1.5958771749e+07
   27        1.3533507617e+11        1.3576596301e+11       -4.3088683723e+08
   28       -3.7893821329e+12       -3.8014469643e+12        1.2064831442e+10
   29        1.0989208185e+14        1.1024196197e+14       -3.4988011183e+11
   30       -3.2967624556e+15       -3.3072588590e+15        1.0496403355e+13
```

4 Part 4. Discussion

Interpretation of Results

The numerical results show two very different facts at the same time.

First, the *exact* sequence is perfectly well-behaved. Since a_n has the integral representation

$$a_n = \int_0^1 x^n e^{x-1}\, dx, \tag{18}$$

it remains positive and decreases toward zero. The theoretical column in the output confirms this: for $n = 30$ the value is still about 3.13×10^{-2}, so the decay is relatively slow but completely regular.

Second, the *direct floating-point recurrence* is extremely sensitive to perturbations. Once the accumulated error reaches a visible level, the recurrence mul-

tiplies it by the factor $-(n+1)$ at the next step. Because this factor grows linearly with n, the total amplification behaves essentially like a factorial. As a result, a harmless machine-level perturbation at the beginning becomes the dominant contribution later, and the computed sequence no longer reflects the true mathematics of the problem.

Why the Forward Recurrence Fails

Suppose the computed value is

$$\widetilde{a}_n = a_n + e_n. \tag{19}$$

Substituting this into the recurrence gives

$$\widetilde{a}_{n+1} = 1 - (n+1)\widetilde{a}_n, \tag{20}$$

while the exact value satisfies

$$a_{n+1} = 1 - (n+1)a_n. \tag{21}$$

Subtracting the two equations yields

$$e_{n+1} = -(n+1)e_n + \delta_n, \tag{22}$$

where δ_n denotes the new round-off introduced at step n. If the propagated part dominates δ_n, then

$$e_n \approx (-1)^n n! e_0. \tag{23}$$

This is exactly the law implemented in the prediction module of the program. The numerical output confirms that this model captures the growth rate and sign alternation of the real error very well once the instability starts to dominate.

Another intuitive explanation is cancellation. For moderate n, the exact product $(n+1)a_n$ is close to 1. Therefore the recurrence computes a_{n+1} as the difference of two nearly equal numbers,

$$a_{n+1} = 1 - (n+1)a_n, \tag{24}$$

which is a classical setting in which significant digits are lost. After that loss of precision occurs, the next multiplication by $(n+1)$ magnifies the remaining error. Hence the recurrence is not backward stable in finite precision arithmetic.

Error Analysis and Prediction

The output of the program provides a clear quantitative picture of the error growth.

- For small n, the error remains close to machine precision and the direct recursion appears trustworthy.
- Around $n = 15$ to 17, the error becomes visible on the scale of the exact answer.
- Near $n = 18$ to 20, the propagated error exceeds the exact value itself, so the numerical result changes sign and then explodes in magnitude.
- After that point, the computed sequence is almost entirely an error-propagation process rather than an approximation to a_n.

The prediction part of the code is especially valuable because it does not merely state that "round-off error exists"; instead, it models the mechanism explicitly. Since the program stores the predicted error in the form

$$M_n = \text{round}(e_n 2^{80}), \tag{25}$$

it can continue tracking the factorial amplification without suffering from the same overflow and precision loss that affect the original recursion. Therefore the comparison between the true error and the predicted error gives strong evidence that the blow-up is caused primarily by the recurrence itself, not by the quadrature reference.

A small discrepancy remains between the predicted and observed errors. This is expected, because the simplified model keeps only the amplified initial perturbation e_0 and does not include every local round-off term δ_n generated along the way. Nevertheless, once the instability dominates, the leading-order model is already sufficient to explain the overall magnitude and alternating sign pattern.

End of Report